

What is JVM :

- * It stands for Java Virtual Machine.
- * It is used to **load and execute** our .class file line by line. [Internally It contains an Interpreter]
- * JVM is used to convert our .class file into appropriate machine code instruction.
- * Along with these, It provides following features
 - a) Loading the .class file
 - b) To verify the .class file [ByteCodeVerifier]
 - c) Allocate the memory for the members of the class
 - d) Delete the un-used objects by Garbage Collector
 - e) Provides Security
 - f) Manage the memory and so on

What is the Difference between Compiler & Interpreter

Compiler (javac)	Interpreter (JVM has Interpreter)
1) Will scan the entire program.	1) Will scan line by line.
2) Will display all the warnings and errors at a time.	2) Will display only one execption (Runtime error) at a time.
3) Will convert S/C into byte code so, separate memory is required to hold the byte code.	3) Interpreter will execute line by line so, separate memory is not requited.
4) Debugging is slow.	4) Debugging is fast.
5) After resolving all the errors & warnings, execution is fast.	5) After resolving the runtime error, Interpreter will again start the execution from line number 1 so, Interpreter is slow in execution .
6) After compilation, we can delete the S/C.	6) We cannot delete byte code due to line by line execution

What is JIT Compiler:

JIT (Just-In-Time) Compiler is a part of the JVM execution engine.

It is used to **enhance the performance** of Java programs.

Internally JVM uses an interpreter, which executes bytecode line by line hence execution is slow.

To overcome this limitation, JVM uses JIT Compiler.

How JIT compiler Works:

When the JVM finds some methods or instructions that are repeated **frequently** (called hotspots), Instead of interpreting them line by line again and again, JIT compiles those bytecode instructions into native machine code.

At the time of execution, JIT compiler directly provides these native machine code to JVM Interpreter for fast execution.

What is data type in java?

It defines the **type of data** that a variable can store.

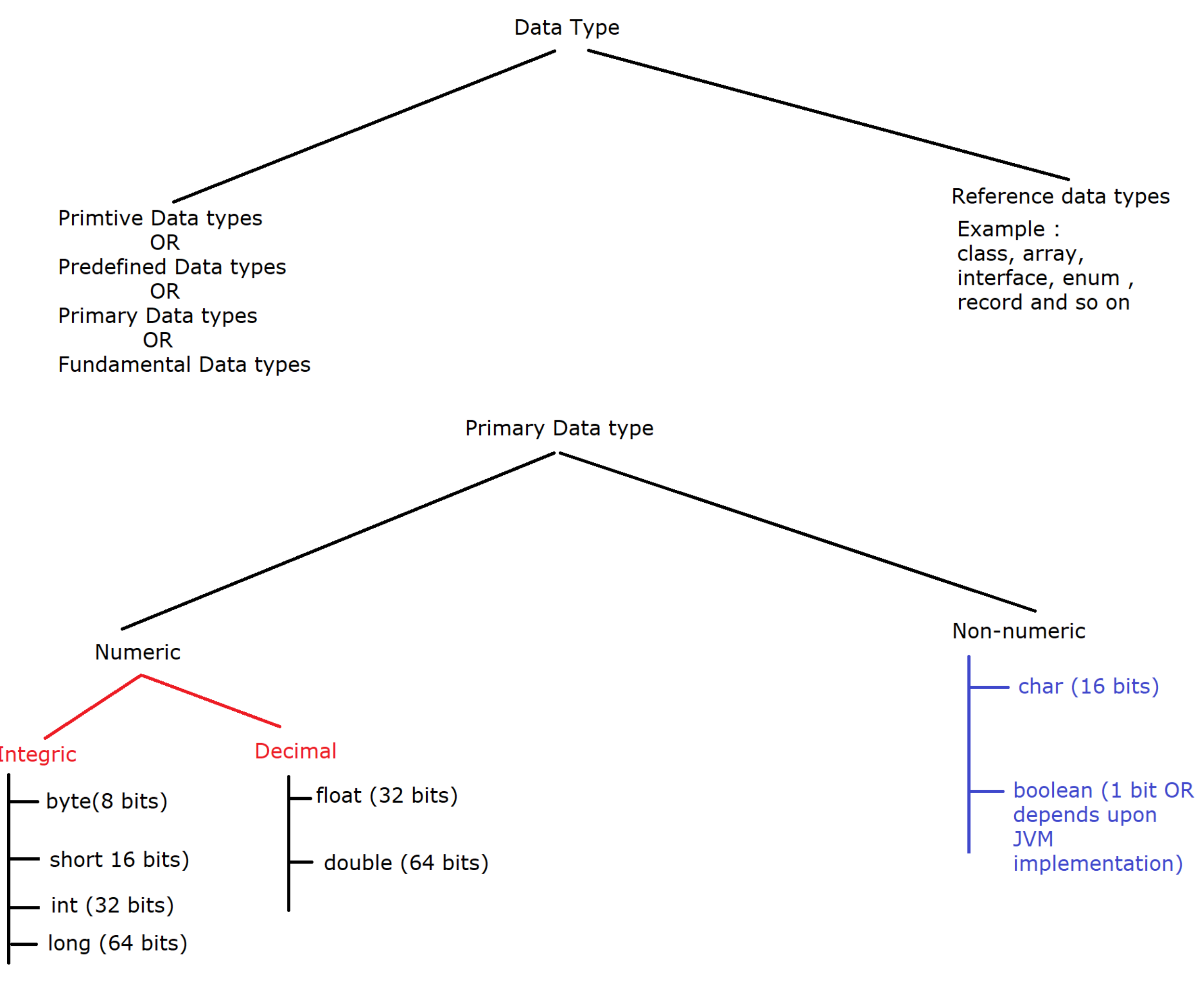
It also describes:

- a) Size → How much memory it occupies in RAM.
- b) Range → What values it can hold (e.g., byte → -128 to 127).
- c) Operations → What kind of operations can be performed (e.g., addition for numbers, concatenation for String).

How many data types in java?

Java data types are mainly divided into two categories:

- 1) Primitive Data type
- 2) Reference Data type [Don't say non-primitive data type]



Note : We have total 8 primitive data types.
Out of 8, **6 data types i.e. byte, short, int, long, float and double represents number data types.**

What is the difference between statically typed and dynamically typed language?

int x; Variable Declaration
x = 10; Variable Initilization

Statically typed language:

The languages where data type is **compulsory** before initialization of a variable are called Statically typed language.
In these languages, once we define the **type of the variable** then it will hold same kind of value till the end of the program.

Example of statically typed languages:
C, C++, JSE, C# and so on

Dynamically typed language:

The languages where Data type **is not** compulsory, it is **optional** to initialize the variable are called Dynamically Typed Language.

In these languages we can provide all different kinds of values to the variable during the execution of the program.

Example of dynamically typed languages:
Visual Basic, JavaScript, Python and so on